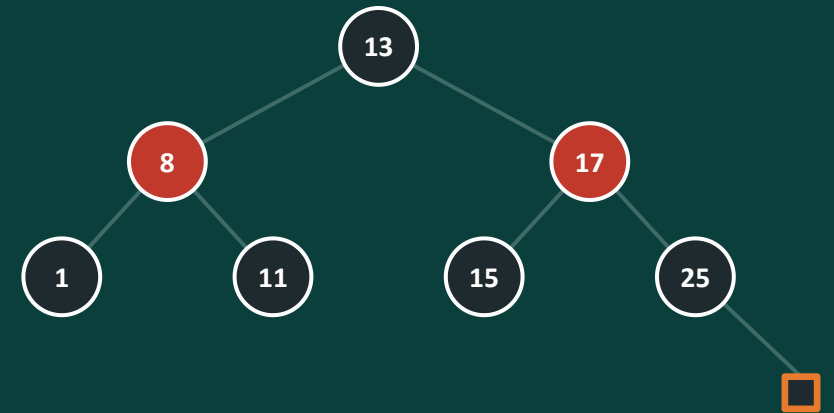




ESTRUTURAS DE DADOS E ALGORITMOS

Árvores Red-Black: Inserção e Eliminação

Aula 9 · tirar um nó sem desequilibrar a árvore

Prof. Doutor Luís Simões da Cunha
luiscunha@ismt.pt

Três perguntas para aquecer

1

De que cor é pintado um nó acabado de inserir? Porquê?

2

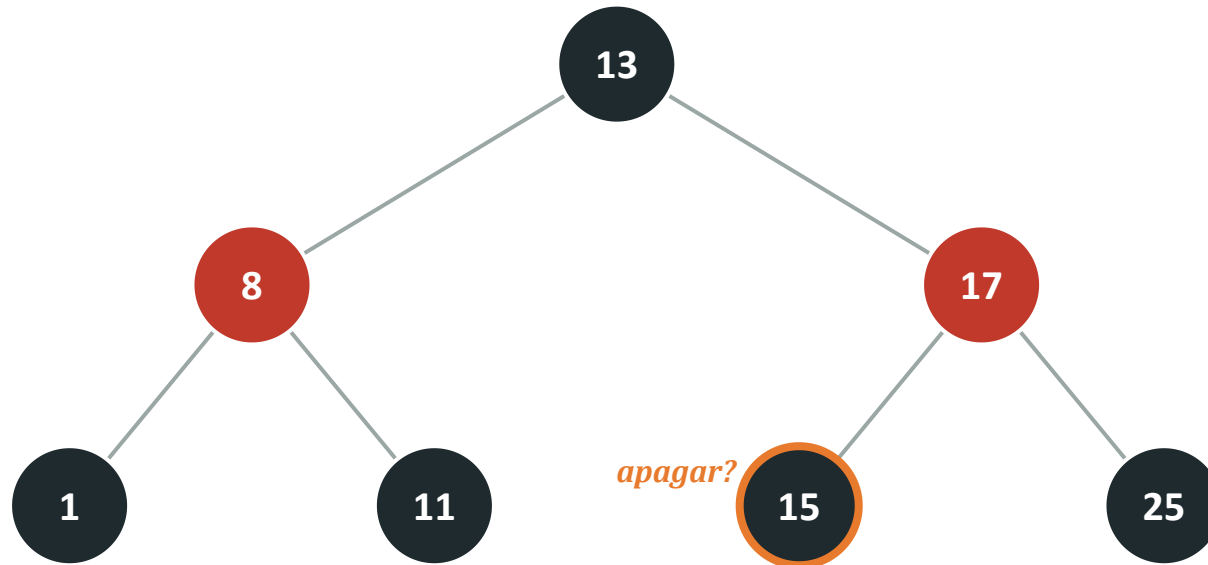
Pai vermelho e tio vermelho: o que se faz?

3

Pai vermelho, tio preto e o novo nó "em triângulo": quantas rotações?

Responda sem olhar para os apontamentos: recordar é o que consolida a memória.

E se apagarmos simplesmente o nó 15? O que se estraga?



Conte os nós pretos da raiz até cada folha.

RESPOSTA

Antes: **2 pretos**
em todos os caminhos.

Depois: o caminho 13 → 17 fica só com
1 preto.

*Regra 5 violada! Apagar um preto deixa
uma "dívida" de um preto.*

Eliminar sem desequilibrar

1

Revisão

Propriedades e inserção, num relance.

2

Remover numa BST

Folha, um filho ou dois filhos.

3

As 5 regras

Quem sai, quem entra e de que cor.

4

Os casos 0 a 4

Pagar a "dívida" de um preto.

5

Exemplos

Sete eliminações, passo a passo.

Pergunta-guia: como tirar um nó preto sem estragar a contagem de pretos?

1

PARTE 1

Revisão

O que já sabemos das árvores Red-Black.

Cinco propriedades, duas que mandam

1 Cada nó é vermelho ou preto.

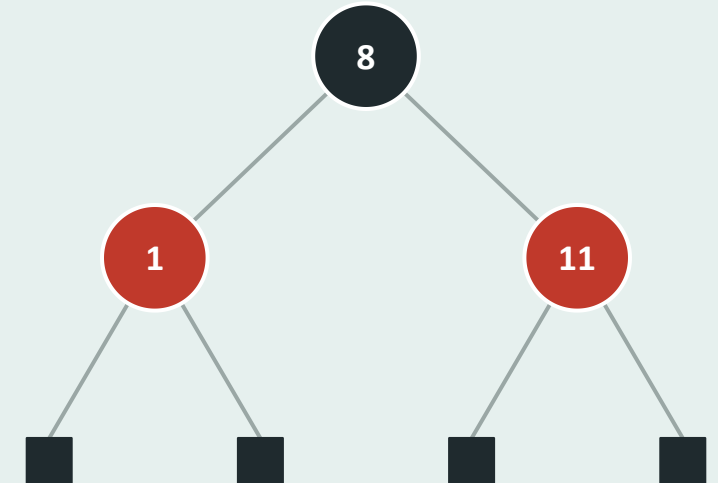
2 A raiz é preta.

3 As folhas NIL são pretas.

4 Um vermelho não tem filhos vermelhos.

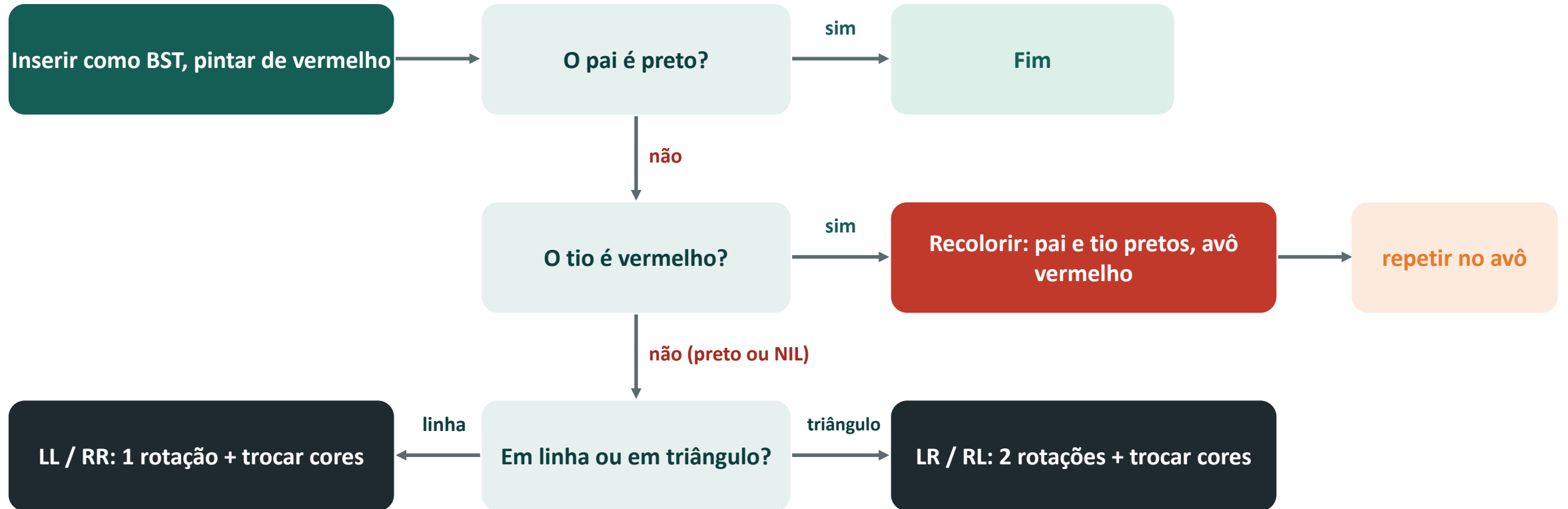
5 Todos os caminhos de um nó até às folhas têm o mesmo número de pretos.

Com folhas NIL



Os quadrados são as folhas NIL ("lugares vazios"). Contam como pretas.

O mapa de decisão da Aula 8



No fim: se a raiz ficou vermelha, pinta-se de preto.

2

PARTE 2

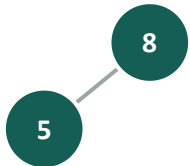
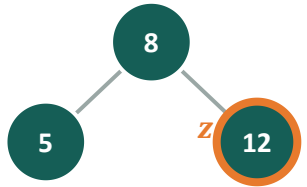
Remover numa BST

Antes das cores, a forma.

Folha, um filho ou dois filhos

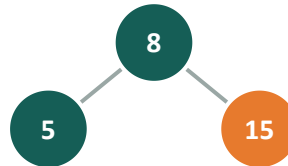
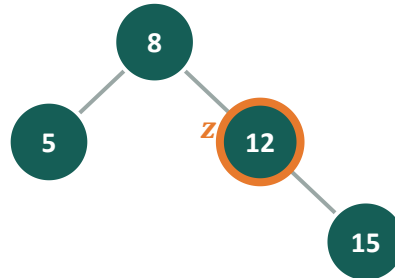
Folha

Remove-se diretamente.



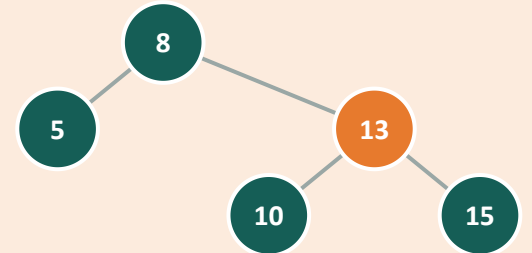
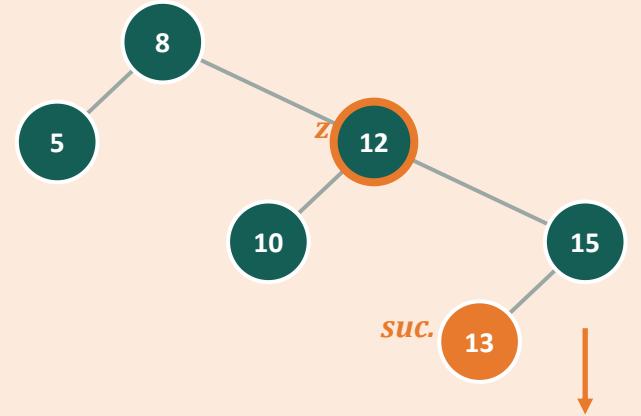
Um filho

O filho sobe para o lugar do nó.

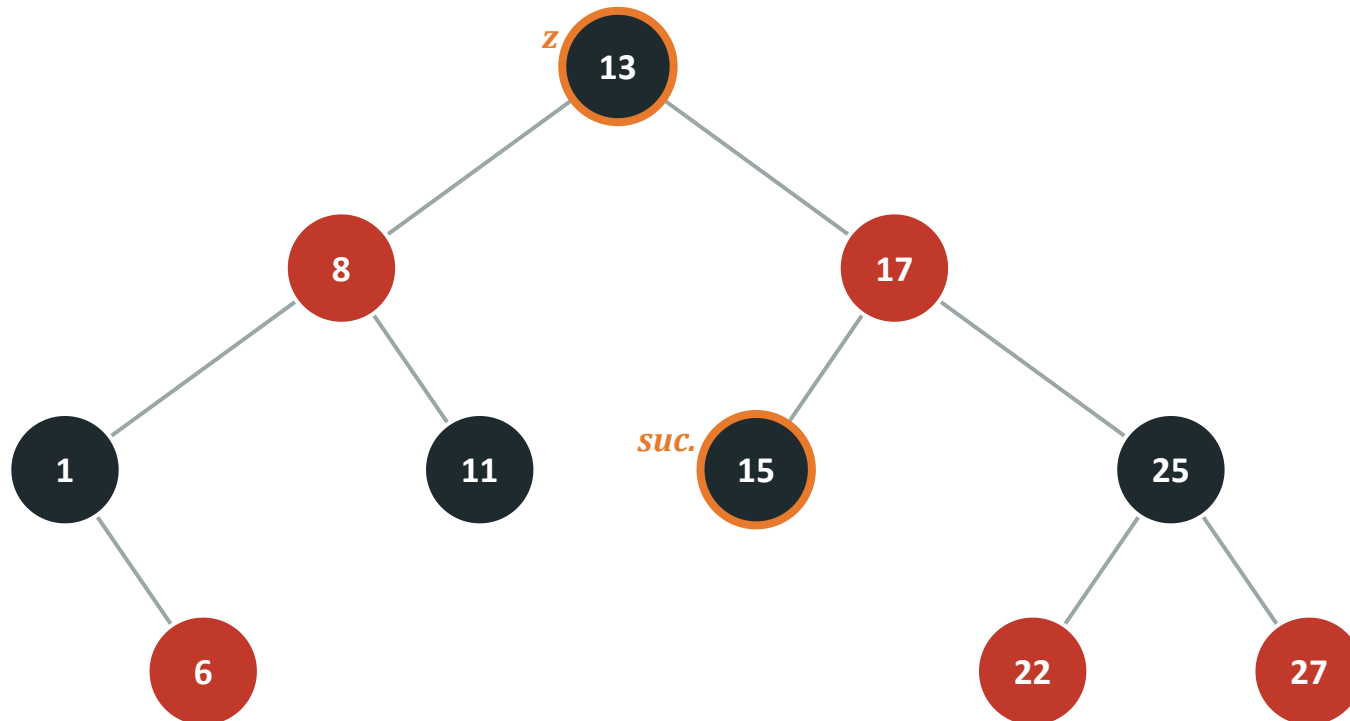


Dois filhos

Troca-se pelo sucessor em ordem e remove-se o sucessor.



Onde está o sucessor?



Eliminar 13

1. Uma vez à direita: 17
2. Sempre à esquerda: 15
3. 15 não tem filho esquerdo

O sucessor é 15.

O sucessor tem, no máximo, um filho (o direito).

3

PARTE 3

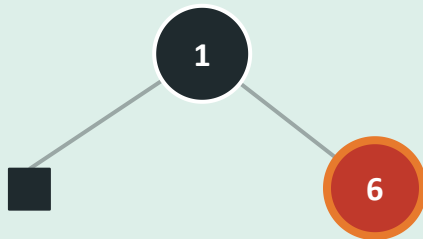
Eliminação Red-Black

Quem sai, quem entra e de que cor.

Vermelho sai à vontade; preto deixa uma dívida

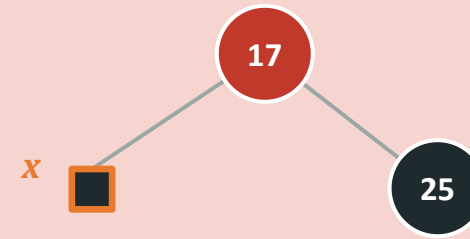
Sai um vermelho

Nenhum caminho perde pretos. Normalmente não há nada a corrigir.



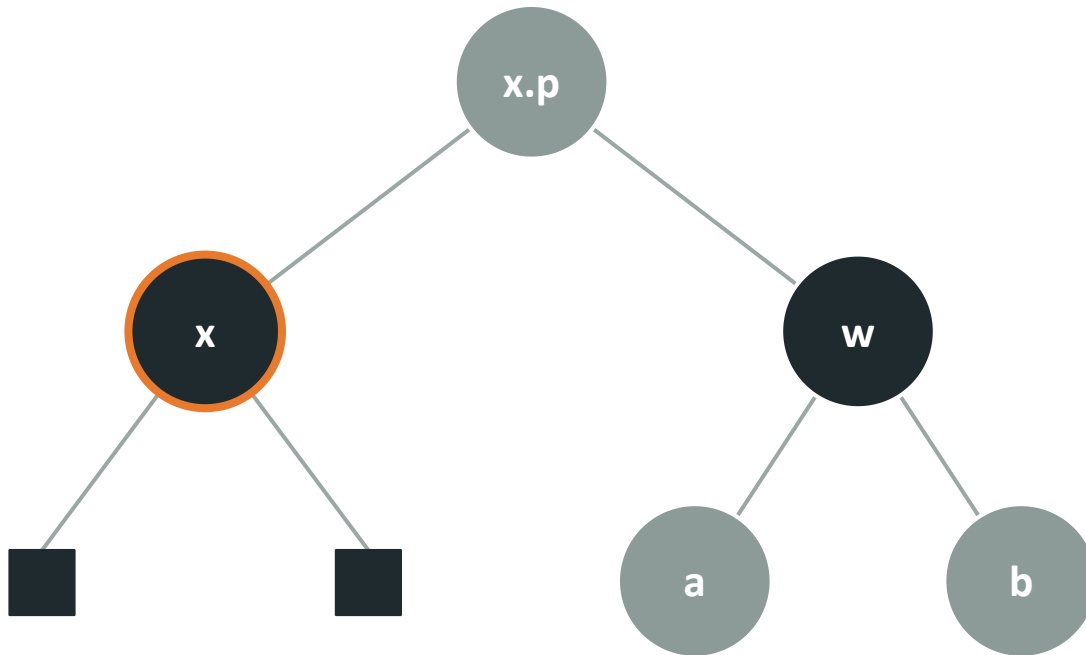
Sai um preto

Um caminho fica com menos um preto. Quem ocupa o lugar — x — fica a "dever" um preto.



Como pagar a dívida? Se x for vermelho, basta pintá-lo de preto. Se for preto, pede-se ajuda ao irmão w — e é daí que vêm os casos 1 a 4.

Quem é quem na eliminação



z

o nó que queremos eliminar

substituto

quem ocupa o lugar de z (com 2 filhos: o sucessor)

x

o nó que fica com a "dívida" de um preto (pode ser NIL)

x.p

o pai de x

w

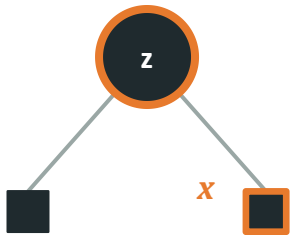
o irmão de x

Cinzento = pode ser de qualquer cor.

Quem é x?

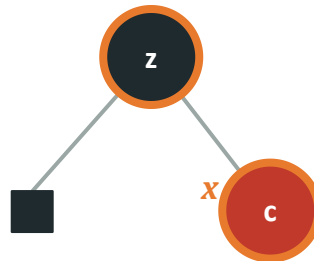
1 z tem 2 filhos NIL

x é NIL



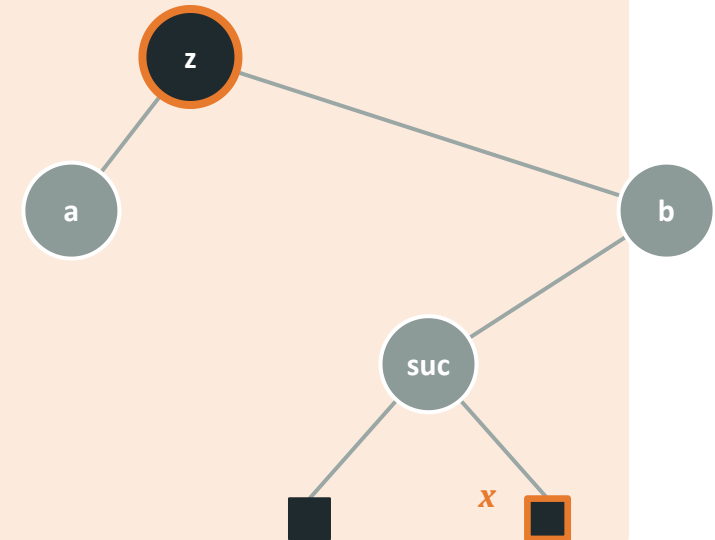
2 z tem 1 filho NIL e 1 não-NIL

x é o filho não-NIL



3 z tem 2 filhos não-NIL

x é o filho direito do sucessor, antes de este ser removido



Olhar para a cor de quem sai e de quem entra

Regra	Nó eliminado	Substituto	Outra condição	O que fazer
1	vermelho	vermelho ou NIL	—	Concluído
2	vermelho	preto (não-NIL)	—	Pintar substituto de vermelho → caso de x
3	preto	vermelho	—	Pintar substituto de preto. Concluído
4	preto	preto	x é a raiz	Concluído
5	preto	preto	x não é a raiz	Ir para o caso de x

Regras 1, 3, 4: fácil — termina logo.

Regras 2 e 5: há uma dívida a pagar → casos.

Que regra se aplica?

A Eliminar um vermelho que é folha **Regra 1**

B Eliminar um preto com um único filho vermelho **Regra 3**

C Eliminar um preto que é folha (e não é a raiz) **Regra 5**

D Eliminar um vermelho com 2 filhos, cujo sucessor é preto **Regra 2**

Tape a coluna da direita e peça a resposta antes de a mostrar.

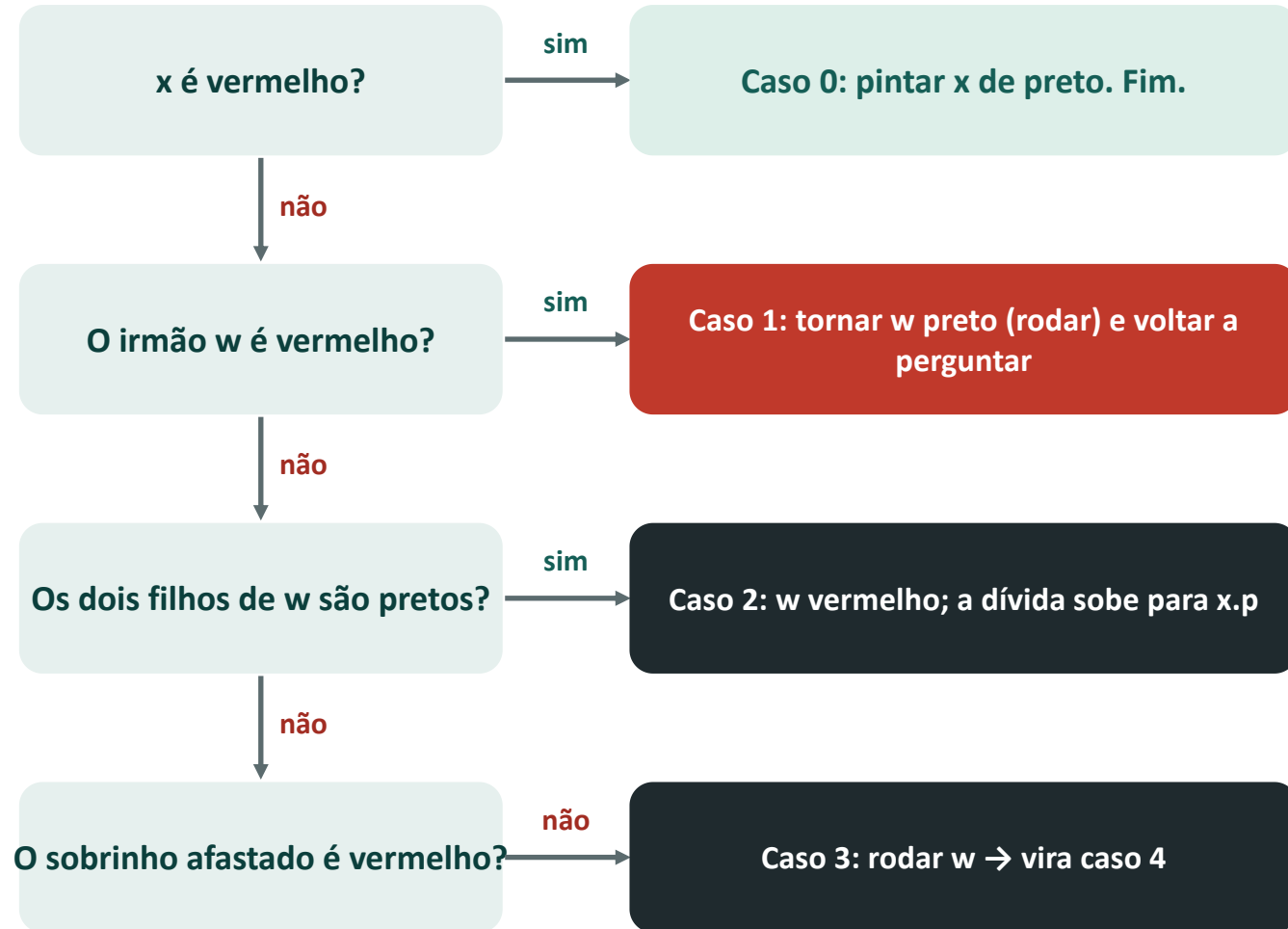
4

PARTE 4

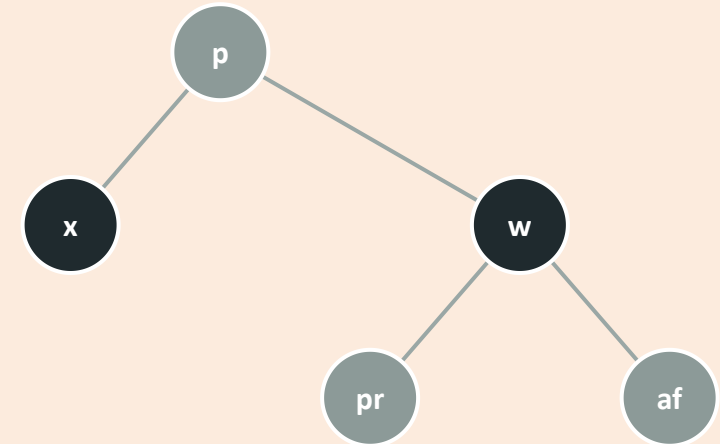
Os casos especiais

Cinco formas de pagar a dívida de um preto.

Que caso se aplica ao nó x?



Sobrinhos



pr = próximo (do lado de x)

af = afastado (lado oposto)

sim (afastado vermelho) → diretamente para o caso 4

Caso 4: rodar x.p e trocar cores. Fim.

x vermelho; ou irmão vermelho

Caso 0

x é vermelho

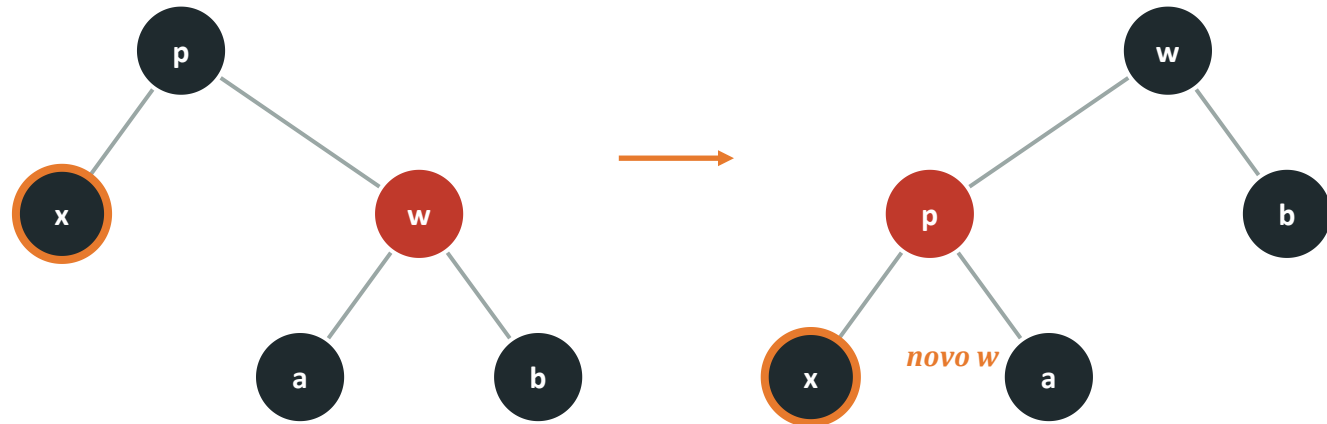


1. Pintar x de preto.

Concluído.

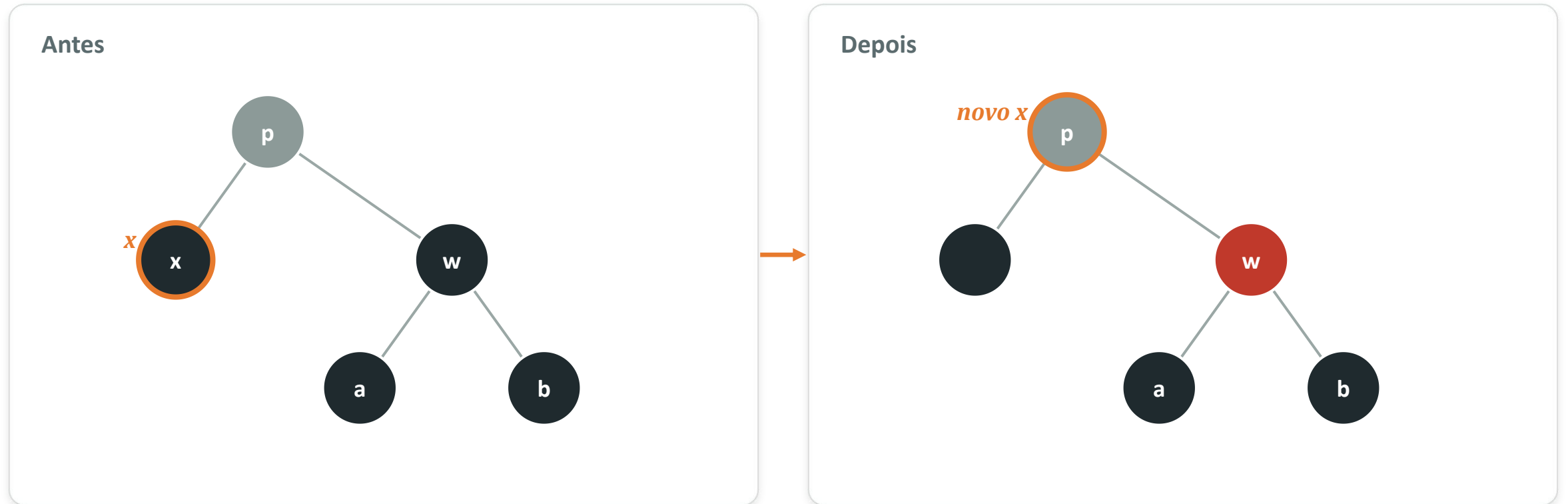
Caso 1

x preto, irmão w vermelho



1. $w \rightarrow$ preto 2. $x.p \rightarrow$ vermelho 3. rodar $x.p$ para o lado de x 4. novo w = irmão de x 5. ir para o caso 2, 3 ou 4

Sobrinhos pretos: a dívida sobe para o pai



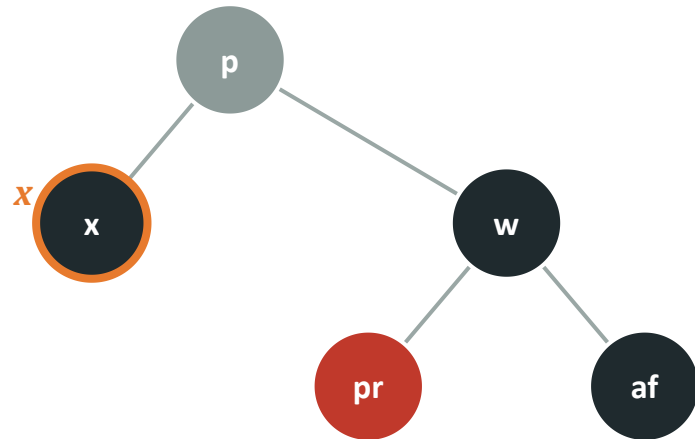
1. Pintar w de vermelho

2. $x = x.p$ (a dívida sobe)

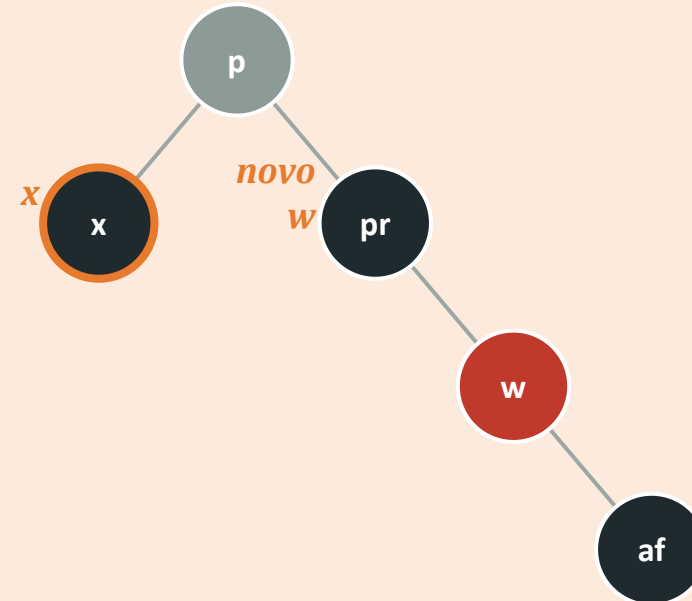
3. x vermelho → caso 0; x raiz → fim; senão → casos 1 a 4

Sobrinho próximo vermelho, afastado preto: endireitar

Antes



Depois: agora é caso 4



1. Pintar o sobrinho próximo de preto

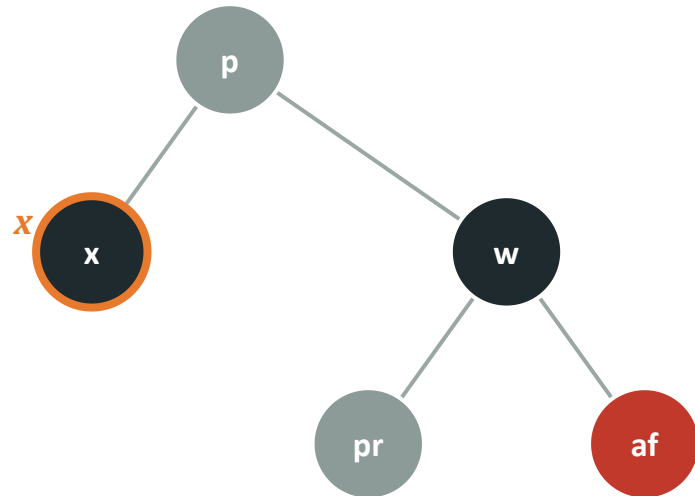
2. Pintar w de vermelho

3. Rodar w (para o lado oposto a x)

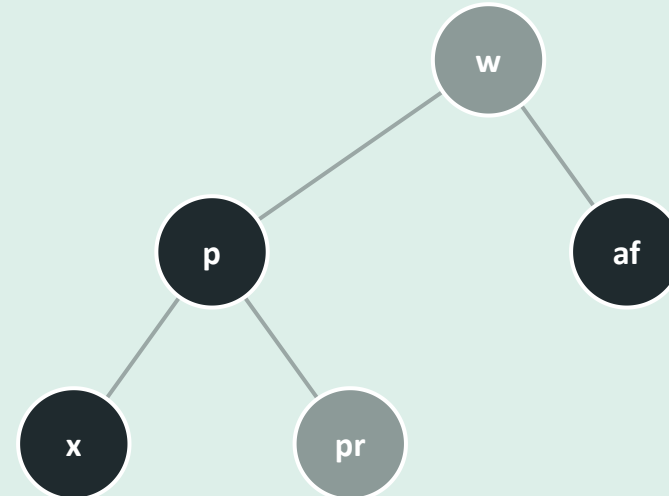
4. Novo w; ir para o caso 4

Sobrinho afastado vermelho: rodar e fechar

Antes



Depois: dívida paga



1. $w \leftarrow \text{cor de } x.p$

2. $x.p \rightarrow \text{preto}$

3. Sobrinho afastado $\rightarrow$ preto

4. Rodar x.p. Fim!

Mesmo com tantos casos, continua $O(\log n)$

	Pesquisar	Inserir	Eliminar
Tempo (pior caso)	$O(\log n)$	$O(\log n)$	$O(\log n)$
Rotações, no máximo	0	2	3
Casos a conhecer	—	3	5 regras + 5 casos

Porque vale a pena? A complexidade dos casos está toda na implementação, feita uma vez. Quem usa um TreeMap em Java tem pesquisa, inserção e eliminação garantidas em $O(\log n)$, sempre.

5

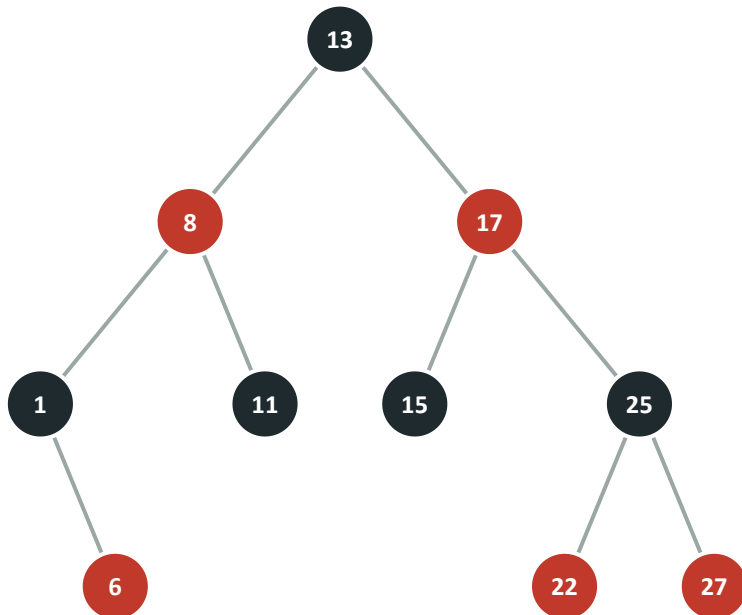
PARTE 5

Exemplos resolvidos

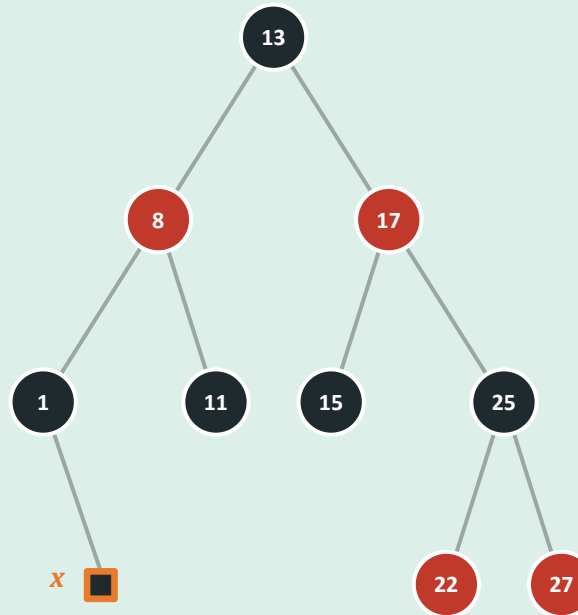
Sete eliminações, passo a passo.

Eliminar 6 (Regra 1) e eliminar 1 (Regra 3)

Árvore de partida

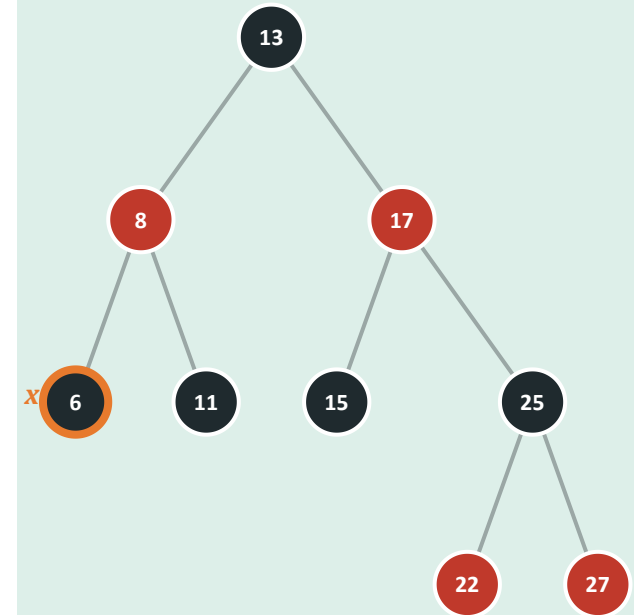


Eliminar 6 Regra 1



6 era vermelho, substituto NIL. Nada a corrigir.

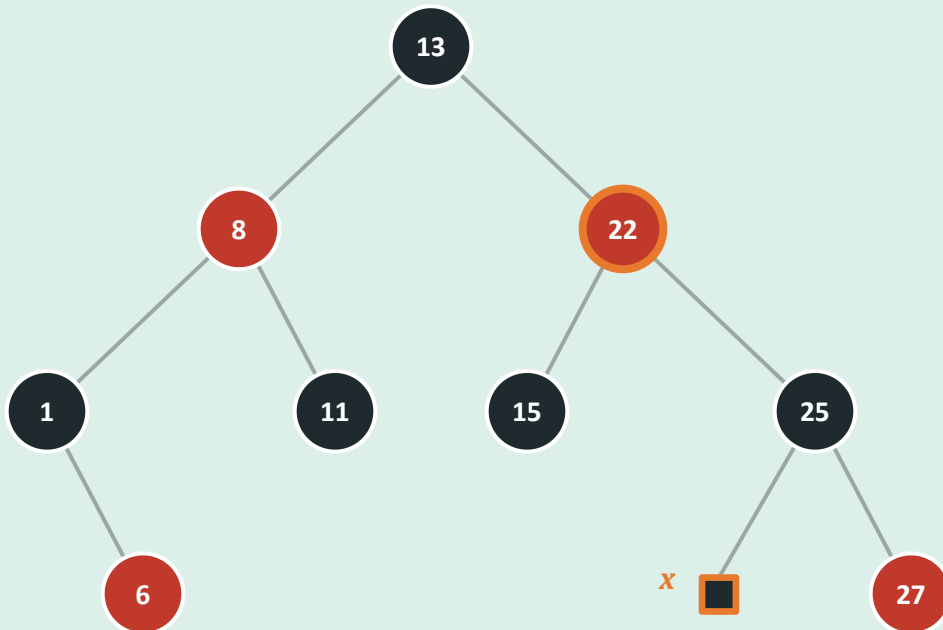
Eliminar 1 Regra 3



1 era preto, substituto 6 vermelho: 6 sobe e fica preto.

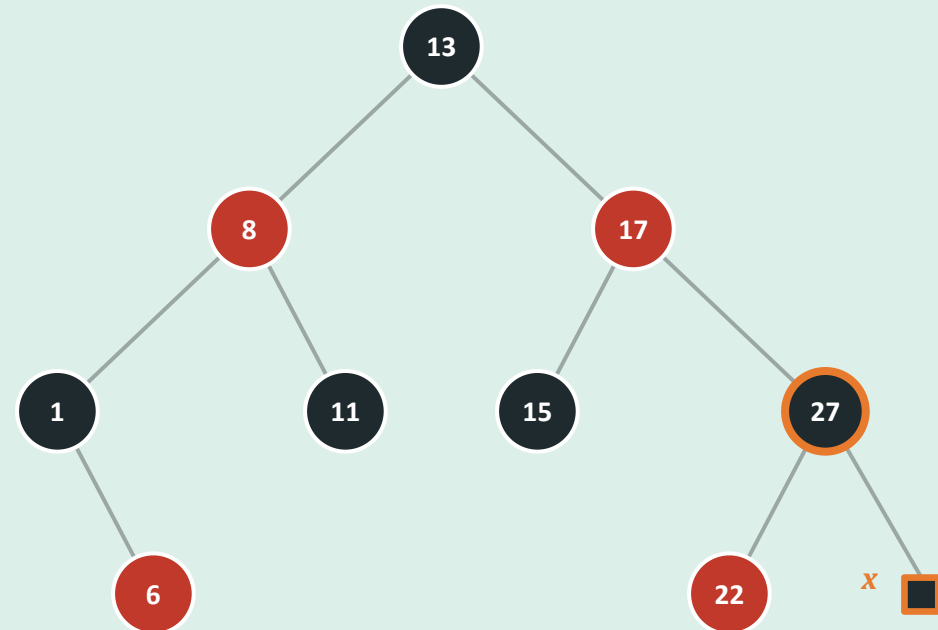
Eliminar 17 (Regra 1) e eliminar 25 (Regra 3)

Eliminar 17 Regra 1 (sucessor 22)



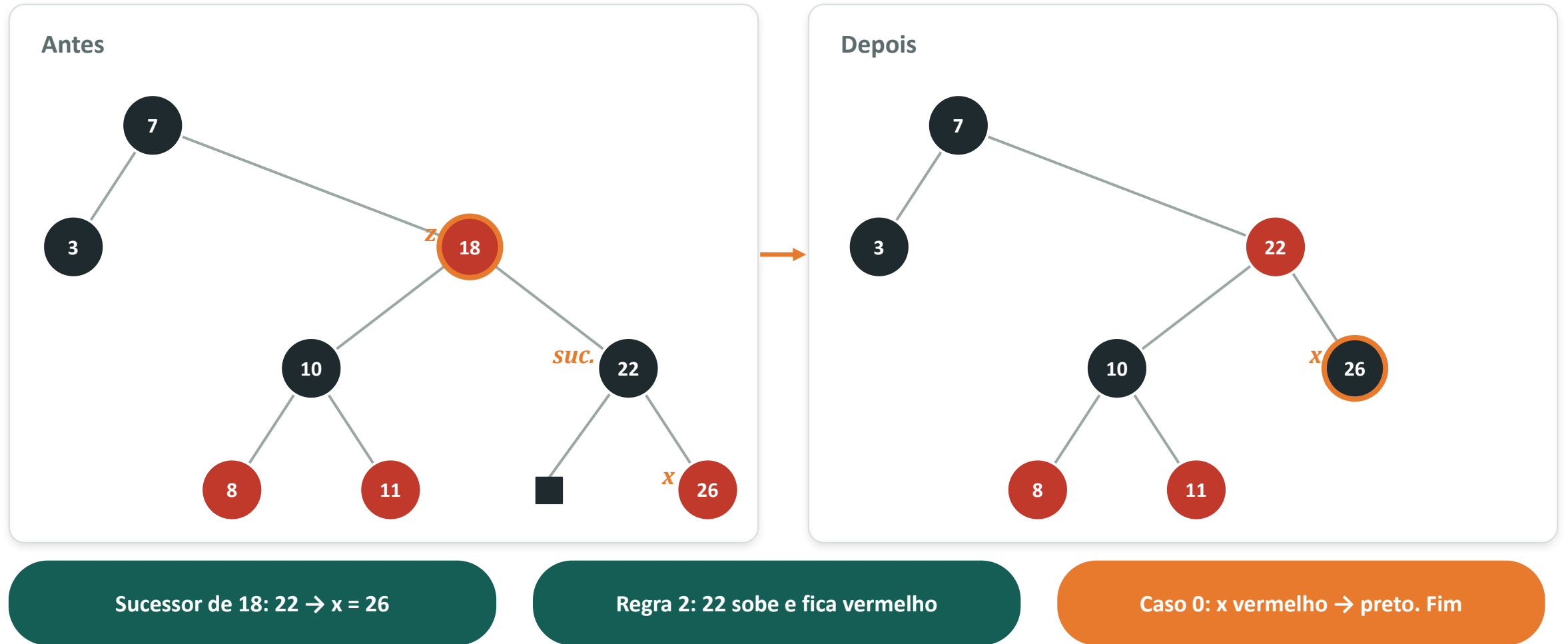
22 ocupa o lugar de 17 e fica vermelho. Nada mais a fazer.

Eliminar 25 Regra 3 (sucessor 27)

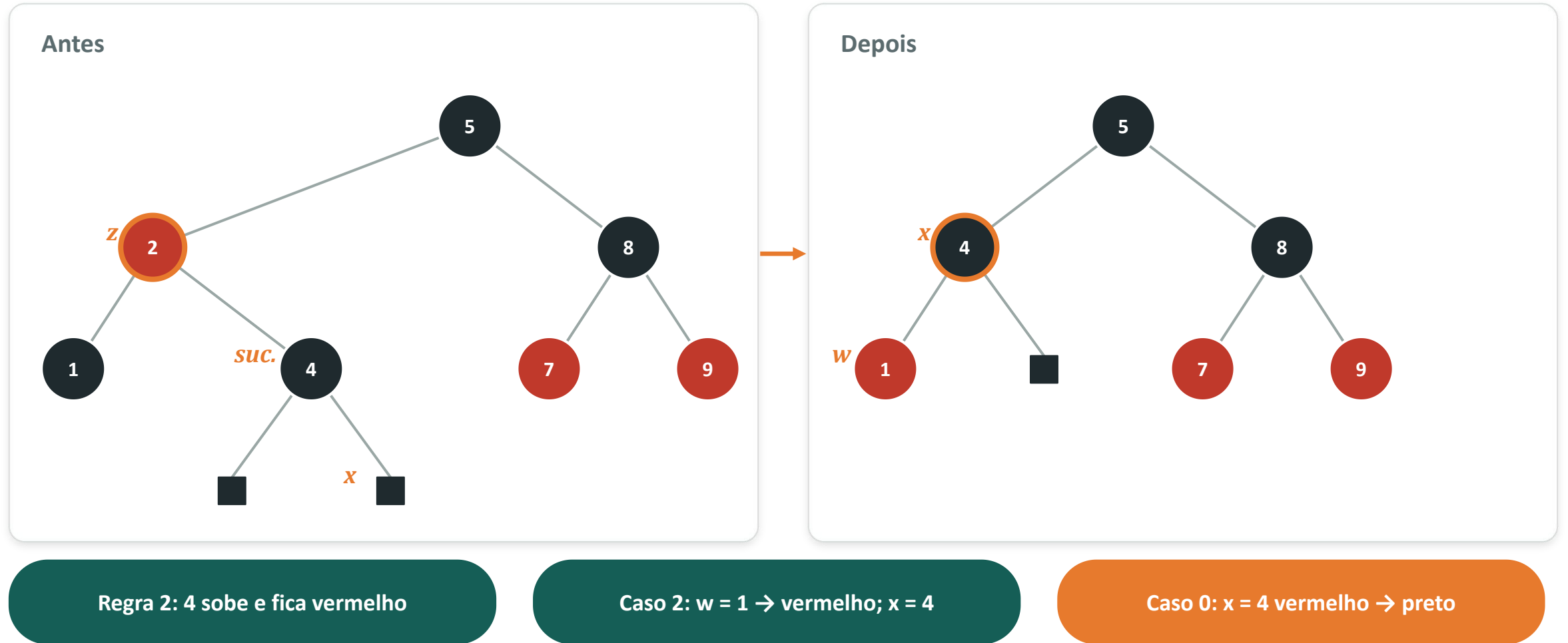


27 ocupa o lugar de 25 e fica preto. Concluído.

Eliminar 18: Regra 2 e caso 0

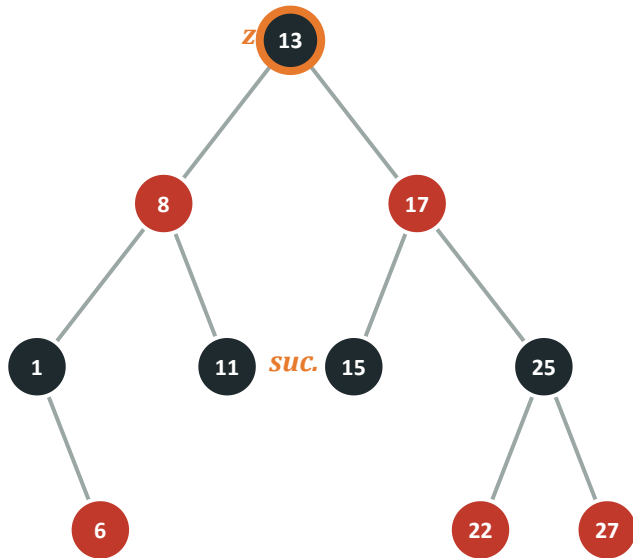


Eliminar 2: Regra 2 e caso 2



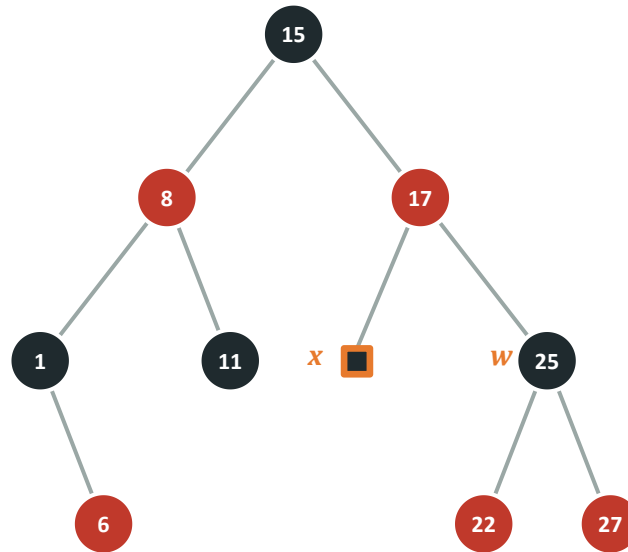
Eliminar 13: Regra 5 e caso 4

Antes



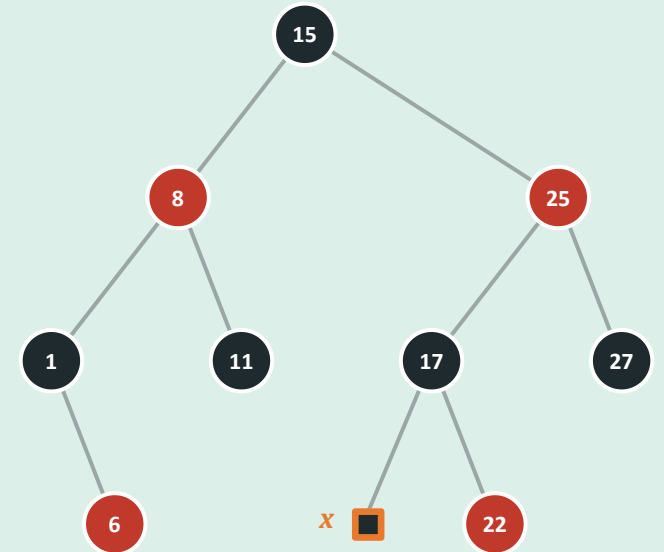
Sucessor 15 $\rightarrow x = \text{NIL}$

Intermédia (Regra 5)



Regra 5: 15 na raiz, x não é raiz

Depois do caso 4



Caso 4: recolorir e rodar 17. Fim

Inserção: retomar a Aula 8

1 Inserir, em papel

30, 86, 76, 8, 60, 13, 66, 61, 17

2 Inserir, em papel

96, 78, 6, 18, 7, 54, 25, 53, 36

3 Analisar o código Java

Implementação disponível no Moodle: testar com os exemplos dos diapositivos.

4 Mostrar a árvore

Método que imprime a árvore com $\vdash$ e $\lhd$ (ver Aula 8).

Agora é a sua vez de eliminar

5

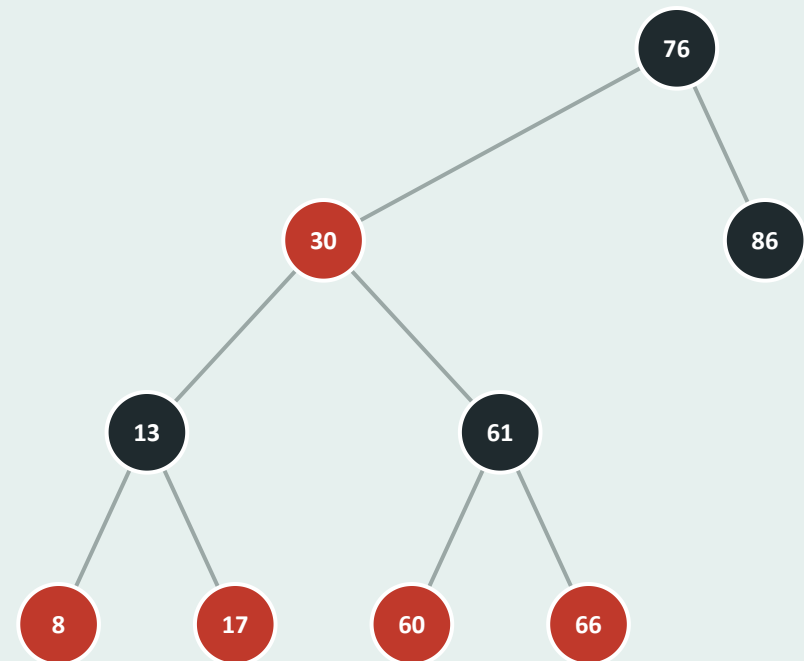
Partindo da árvore do exercício 1, elimine, por esta ordem:



Em cada eliminação indique a regra e os casos especiais que usou.

Dica: um dos valores obriga a passar por três casos seguidos. Qual?

Árvore de partida (ex. 1)



Confirme no simulador de Galles.

Três ideias a reter

1

Primeiro, a BST

Folha ou um filho: remove-se. Dois filhos: troca-se pelo sucessor em ordem.

2

Depois, as cores

Sai um vermelho: tudo bem. Sai um preto: x fica a dever um preto (regras 2 e 5).

3

Pagar a dívida

x vermelho $\rightarrow$ preto (caso 0). Senão, pede-se ajuda ao irmão w (casos 1 a 4).

Pesquisar, inserir e eliminar: sempre $O(\log n)$.

Bibliografia

1

Weiss, M. A. (2012). *Data Structures and Algorithm Analysis in Java* (3.^a ed.). Pearson.

2

Chang, A. *red-black-deletion-steps*. GitHub. github.com/alenchang/red-black-deletion-steps

3

Galles, D. *Red/Black Tree Visualization*. University of San Francisco. cs.usfca.edu/~galles/visualization/RedBlack.html

Dúvidas? Prof. Doutor Luís Simões da Cunha · luiscunha@ismt.pt